Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №5

Дисциплина: Основы программирования на Python

Тема: Функциональные возможности Python: функции высшего порядка, итераторы и генераторы

Вариант 2: Сотрудники

Цель работы

Получить практические навыки использования функциональных возможностей языка Python для обработки коллекций данных и построения последовательностей. В процессе выполнения работы необходимо освоить функции как объекты первого класса, функции высшего порядка, анонимные функции lambda, встроенные функции map(), filter() и reduce(), сортировку sorted() с ключом, функции any() и all(), включения (comprehensions), генераторы с оператором yield, а также итераторы и ленивые вычисления.

Постановка индивидуального задания (Вариант 2. Сотрудники)

Разработать консольное приложение обработки набора объектов предметной области «Сотрудники». Для сотрудника хранить ФИО, отдел, должность и зарплату. Реализовать фильтрацию по зарплате, получение списка должностей, сортировку по зарплате, поиск самого высокооплачиваемого сотрудника, проверку наличия сотрудников определённого отдела и генератор сотрудников выбранного отдела. Согласно общим требованиям программа должна содержать не менее одного базового и двух производных классов, коллекцию объектов, минимум две функции высшего порядка, две lambda-функции, вызовы map(), filter() и sorted() с ключом, включение, генератор, проверки any() и all(), собственный итератор и обработку исключительных ситуаций.

Краткие теоретические сведения

В Python функции являются объектами первого класса: их можно присваивать переменным, передавать в качестве аргументов и возвращать из других функций. Функция, которая принимает другую функцию как параметр или возвращает её как результат, называется функцией высшего порядка. Такой подход позволяет отделить общий алгоритм обработки коллекции от конкретного действия над элементом.

Анонимная функция lambda позволяет объявить небольшую функцию в одну строку без имени: lambda emp: emp.salary. Она оправдана в случаях, когда функция нужна однократно как аргумент другой функции. Встроенная функция map() применяет переданную операцию к каждому элементу и возвращает итератор, filter() оставляет только элементы, удовлетворяющие предикату, reduce() из модуля functools сворачивает коллекцию в одно значение, а sorted() с параметром key задаёт правило сравнения элементов.

Функции any() и all() проверяют логическое условие для коллекции: any() возвращает True, если условию удовлетворяет хотя бы один элемент, all() — только если условию удовлетворяют все элементы. Включения (list, set и dict comprehensions) позволяют компактно построить новую коллекцию на основе существующей.

Генератор — это функция, содержащая оператор yield. При вызове она не выполняет тело сразу, а возвращает объект-генератор, который выдаёт значения по одному при каждом обращении. Благодаря этому генератор не хранит всю последовательность в памяти, что и обеспечивает ленивые вычисления. Итератор — объект с методами __iter__() и __next__(), который возвращает очередной элемент и возбуждает исключение StopIteration при завершении перебора.

Структура проекта

Программа состоит из четырёх модулей. Модуль models.py содержит иерархию классов предметной области, operations.py — функции обработки коллекций, generators.py — генератор и собственный итератор, main.py — консольное меню с демонстрацией каждой конструкции.

Файл	Содержимое
models.py	Базовый класс Person, производные Employee и Manager, переопределение метода __str__()
operations.py	filter_objects(), transform_objects(), sort_objects(), find_max_salary(), has_department(), all_salaries_positive(), get_positions(), total_salary()
generators.py	Функция-генератор employee_generator() с оператором yield и класс-итератор EmployeeIterator
main.py	Коллекция из шести сотрудников и меню из одиннадцати пунктов с обработкой исключений

Иерархия классов предметной области

Базовый класс Person хранит имя сотрудника и реализует метод __str__(). Класс Employee наследует Person и добавляет отдел, должность и зарплату, вызывая конструктор базового класса через super(). Класс Manager, в свою очередь, наследует Employee и дополняет объект премией. Такая иерархия из трёх уровней обеспечивает повторное использование кода и демонстрирует полиморфное поведение метода __str__(): каждый класс формирует собственную строку вывода.

Листинг 5.1 — Иерархия классов (models.py)

```
class Person:
    def __init__(self, name):
        self.name = name


    def __str__(self):
        return f"{self.name}"


class Employee(Person):
    def __init__(self, name, department, position, salary):
        super().__init__(name)
        self.department = department
        self.position = position
        self.salary = salary


class Manager(Employee):
    def __init__(self, name, department, position, salary, bonus):
```

```
        super().__init__(name, department, position, salary)
        self.bonus = bonus
```

Функции высшего порядка и lambda-выражения

Модуль operations.py построен на передаче функций в качестве аргументов. Функция filter_objects(objects, predicate) принимает предикат и возвращает результат filter(), функция transform_objects(objects, operation) применяет операцию через map(), a sort_objects(objects, key_function) выполняет сортировку с переданным ключом. Благодаря этому одна и та же функция обработки используется с разными условиями: в консольном меню в неё передаются lambda-выражения lambda e: e.salary > min_salary и lambda e: e.salary.

Функция find_max_salary() находит сотрудника с наибольшей зарплатой через max() с ключом lambda emp: emp.salary, функция get_positions() получает список должностей через map() и lambda, а функция total_salary() сворачивает коллекцию в сумму с помощью reduce(). Проверки has_department() и all_salaries_positive() построены на генераторных выражениях внутри any() и all().

Листинг 5.2 — Функции высшего порядка (operations.py)

```
def filter_objects(objects, predicate):
    return filter(predicate, objects)


def sort_objects(objects, key_function):
    return sorted(objects, key=key_function)


def find_max_salary(objects):
    if not objects:
        return None
    return max(objects, key=lambda emp: emp.salary)


def has_department(objects, department):
    return any(emp.department == department for emp in objects)


def all_salaries_positive(objects):
    return all(emp.salary > 0 for emp in objects)


def total_salary(objects):
    if not objects:
        return 0
    return reduce(lambda acc, emp: acc + emp.salary, objects, 0)
```

Генераторы, итераторы и ленивые вычисления

Функция employee_generator(objects, department) содержит оператор yield и выдаёт сотрудников указанного отдела по одному, не создавая промежуточный список. Собственный итератор EmployeeIterator хранит коллекцию и текущий индекс, метод __iter__() возвращает сам объект, а метод __next__() выдаёт очередной элемент и возбуждает StopIteration после исчерпания коллекции, что позволяет использовать объект в цикле for.

Для демонстрации ленивых вычислений в меню предусмотрен отдельный пункт: создаются список и генераторное выражение из миллиона квадратов чисел, после чего сравниваются размеры объектов в памяти с помощью sys.getsizeof(). Список занимает десятки мегабайт, тогда как генератор — несколько сотен байт, поскольку значения вычисляются по требованию.

Листинг 5.3 — Генератор и собственный итератор (generators.py)

```
def employee_generator(objects, department):
    for emp in objects:
        if emp.department == department:
            yield emp


class EmployeeIterator:
    def __init__(self, objects):
        self.objects = objects
        self.index = 0


    def __iter__(self):
        return self


    def __next__(self):
        if self.index >= len(self.objects):
            raise StopIteration
        result = self.objects[self.index]
        self.index += 1
        return result
```

Раздел тестирования

Проверка работоспособности выполнялась вручную через консольное меню на подготовленной коллекции из шести сотрудников: четырёх специалистов и двух руководителей. Каждый пункт меню соответствует отдельной конструкции языка, поэтому проверка охватила все требования задания.

№	Действие в меню	Проверяемая конструкция и результат
1	Показать всех сотрудников	Полиморфный вызов __str__() для объектов Employee и Manager
2	Фильтр по зарплате больше 70000	filter() с lambda-предикатом, выведены сотрудники IT и Sales
3	Список должностей	map() с lambda, получен список из шести должностей
4	Сортировка по зарплате	sorted() с ключом lambda e: e.salary
5	Самый высокооплачиваемый	max() с ключом, найден тимлид с зарплатой 120000
6	Проверка наличия отдела	any() по генераторному выражению
7	Проверка положительных зарплат	all() по генераторному выражению
8	Генератор сотрудников отдела	yield, поочерёдная выдача сотрудников IT
9	Собственный итератор	методы __iter__() и __next__(), завершение по StopIteration
10	Сравнение списка и генератора	sys.getsizeof(): генератор занимает во много раз меньше памяти
11	Общая сумма зарплат	reduce() с lambda, накопительная сумма

Дополнительно проверялась обработка некорректного ввода: при вводе текста вместо числа в пункте фильтрации перехватывается исключение ValueError и выводится сообщение об ошибке, после чего меню продолжает работу. Ввод несуществующего отдела в пунктах 6 и 8 приводит к корректному сообщению об отсутствии сотрудников без аварийного завершения программы.

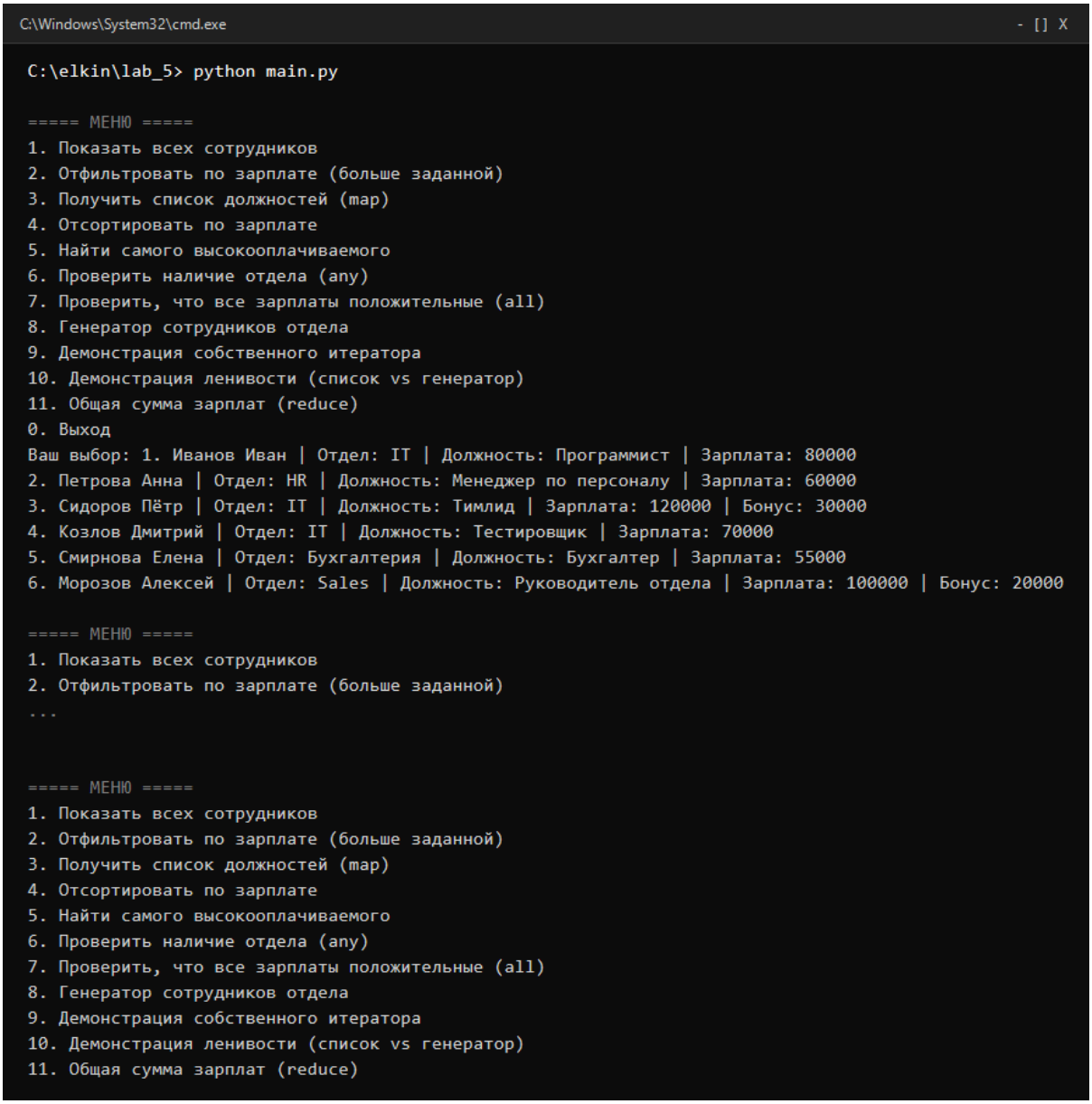


Рисунок 5.1 — Работа программы: список сотрудников, фильтрация по зарплате и сумма зарплат

Листинги ключевых фрагментов программы

Листинг 5.4 — Передача lambda-функции в функцию высшего порядка (main.py)

```
min_salary = float(input("Введите минимальную зарплату: "))
filtered = list(filter_objects(employees, lambda e: e.salary > min_salary))
print_employees(filtered)


positions = get_positions(employees)
print("Должности:", positions)
```

Листинг 5.5 — Демонстрация ленивых вычислений (main.py)

```
list_result = [x * x for x in range(1_000_000)]
print("Размер списка в памяти (байт):", sys.getsizeof(list_result))


gen_result = (x * x for x in range(1_000_000))
print("Размер генератора в памяти (байт):", sys.getsizeof(gen_result))
```

Листинг 5.6 — Обработка исключений в меню (main.py)

```
try:
    ...
except ValueError:
    print("Ошибка: введено некорректное значение. "
          "Пожалуйста, введите число.")
except Exception as e:
    print(f"Произошла ошибка: {e}")
```

Вывод

В ходе выполнения лабораторной работы освоены функциональные возможности языка Python. Разработанное приложение демонстрирует передачу функций как аргументов, применение lambda-выражений, встроенных функций map(), filter(), reduce(), sorted(), any() и all(), а также включений. Реализованы собственный генератор и итератор, показано различие между немедленным созданием списка и ленивыми вычислениями. Использование функций высшего порядка позволило отделить алгоритм обхода коллекции от конкретной операции над сотрудником, что делает код короче и универсальнее.